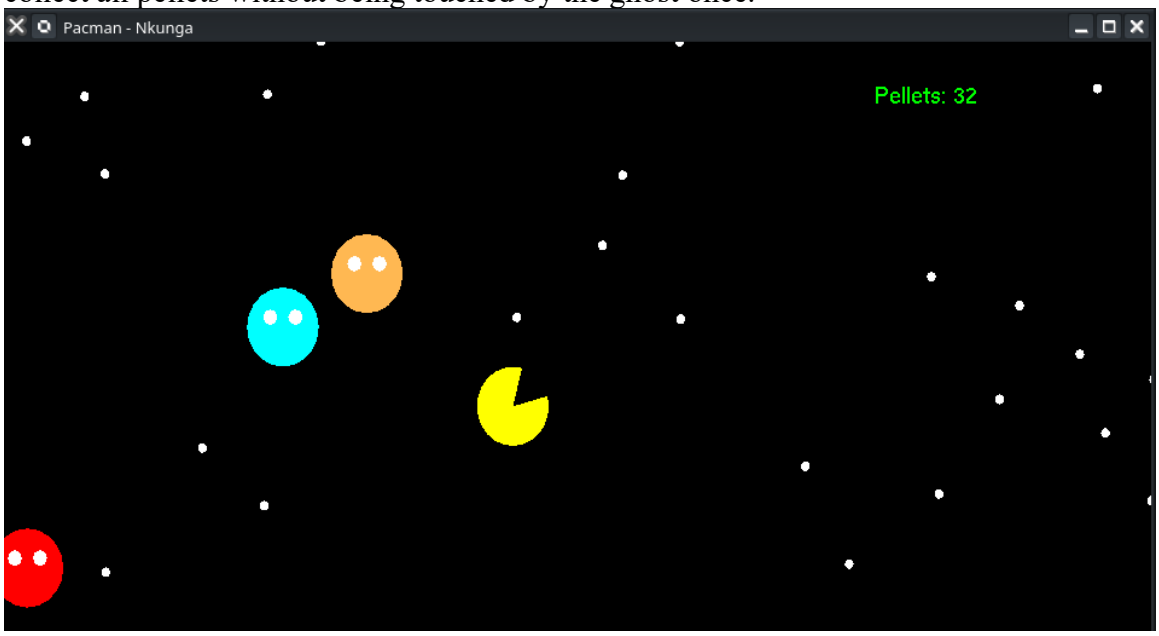
Programming Project Report

Daniel Nkunga
100292653

Problem Statement:

The goal of this program was to implement OpenGL's callback function to allow for player input to influence a program. This is done through an interactive game using (some) characters from the classic video game Pac-Man. The goal of the player is to collect all pellets without being touched by the ghost once.



Design:

The building block of this program was the last programming project where a basic polygon class was implemented and the concepts of inheritance. Key to this project, all objects were implemented as circles, more accurately highly sided polygons, and each had a central point named position assigned to them.

The Pac-Man object was the first implemented and one of the most complex due to its mouth. It is a twenty sided polygon with a certabin amount carved out for its mouth. It was also the first class implemented. It served as the basis for all other programs and established how to implement a circle by translating values into radians and generating coordinates.

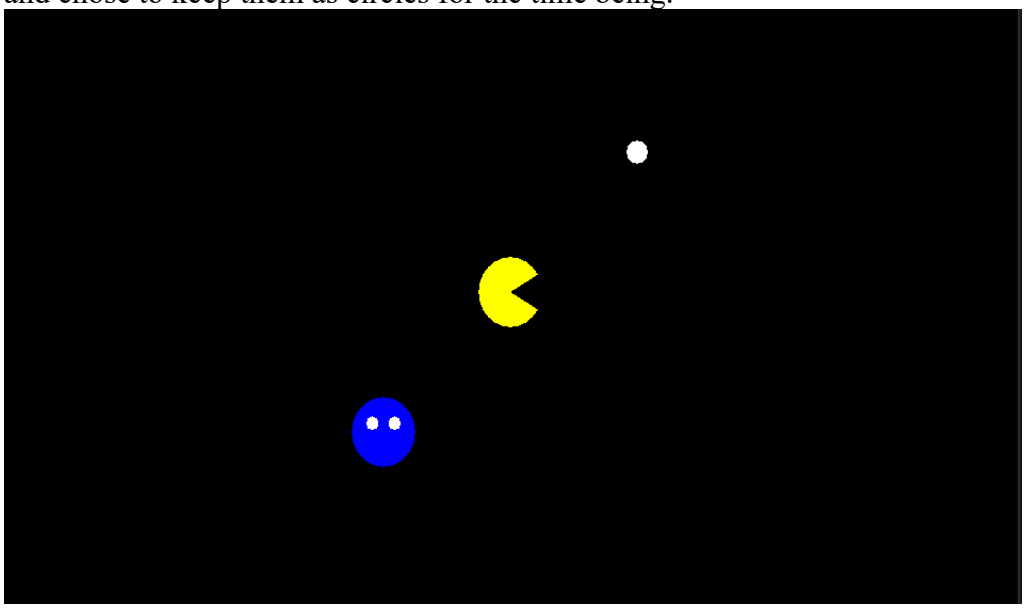
The pellet class was the next implemented and was the simplist to make. They are just a "circle" (twelve sided polygon) that grew smaller and smaller as the project went on. The most complex feature about them came later in development with their update function. Pellets bounce off walls and slowly loose speeds with each bounce.

The ghost class's complications mainly came through thier AI. Ghost are implemented as circles that go through walls and move in random directions.

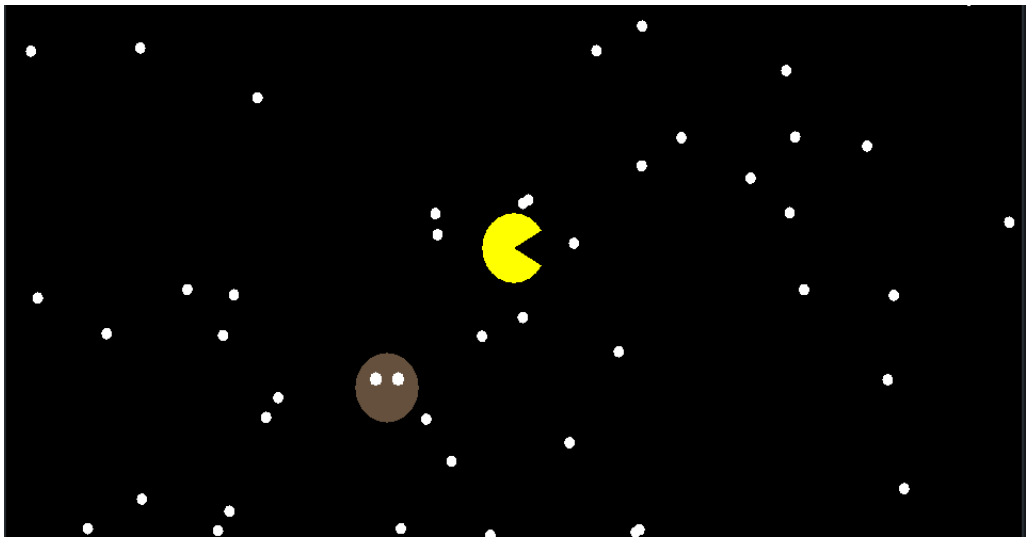
Implementation:

As stated above, this code used the previous project as stater code. This not only allowed me to reuse the polygon class, but also allowed me to keep my debugging functions and a working main for testing.

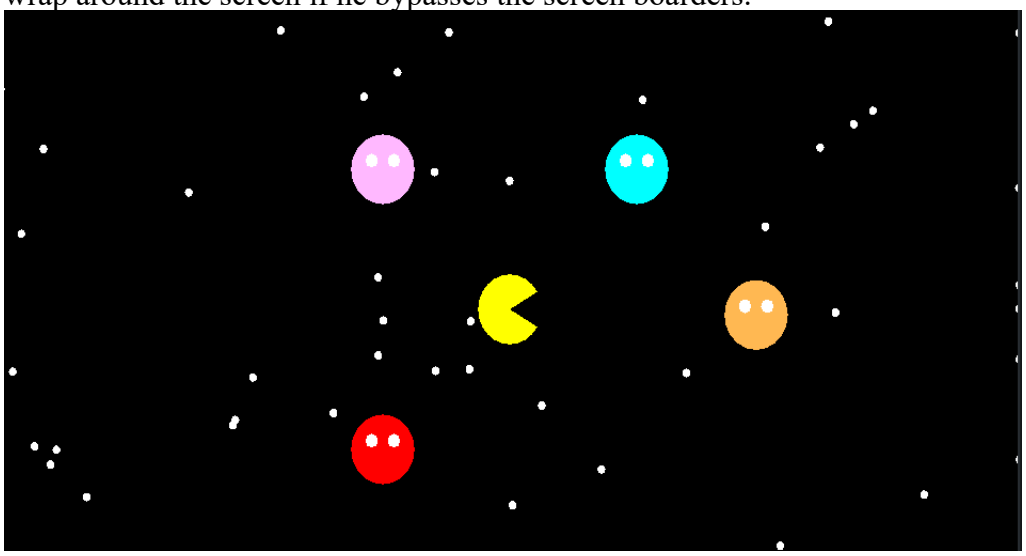
The implementation of this project first came with rendering everything to the screen first. Because I couldn't find a native way to implement circles in OpenGL, I followed a Reddit thread [1] to implement simple circles as high sided polygons. Pacman's rendering was the most complicated due to its mouth, but, due to the way the polygon class was implemented, it simplifies to starting to draw Pac-Man at the top of his mouth and ending at the back of this throat, allowing OpenGL's polygon rendering functionality to handle the rest. The pellets were a simple extension of this concept and implemented in a similar way. Initially, ghost were implemented more faithful to their canonical design, but I had decided to use circle based collision detection for all objects and chose to keep them as circles for the time being.



After getting everything to screen, I focused on getting the pellets moving across the screen. I had the pellets bounce off walls and very slightly loose speed with each bounce (in the hope the players could have an easier time catching them as the game went on.



After pellets, ghost and ghost logic was implemented. I started with coding the AI for Clide because, in the base game, he just moves at random. Due to poor time management on my end, it also allowed me to copy his AI to the other ghost. Clide's AI allows him to periodically pic directions and begin moving in them. He is also allowed to wrap around the screen if he bypasses the screen boarders.

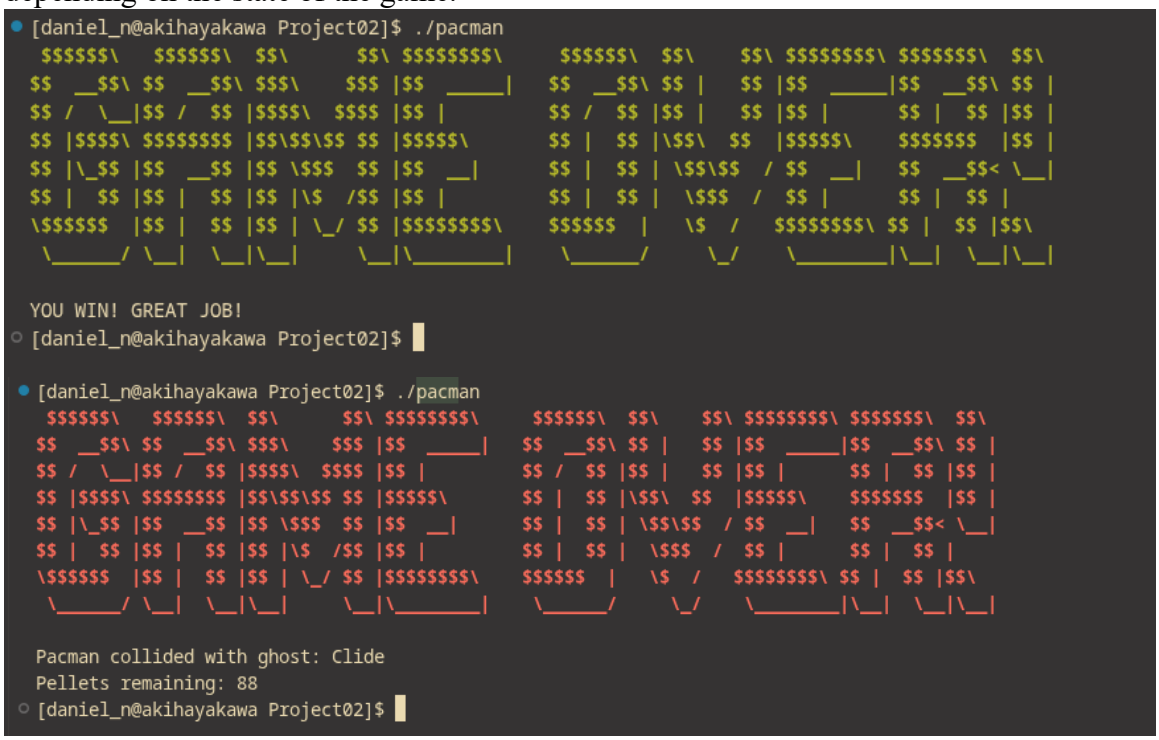


Following ghost movement and "intelligence," I worked on getting Pac-Man to move. This was surprisingly simple using the example code of "keyboard.cpp" as a jumping off point. Pac-Man is able to use both the arrow-key and WASD to move.

After this came collision detection. As stated earlier, I wanted to keep everything as circles to allow me to use a simple collision detection function I remembered from a YouTube [2] video. By just comparing the distance between the center of two circles against the sum of their radii, you are able to determine if they are colliding very cheaply computationally. It also allowed me to get the project to a minimum viable state the fastest by allowing me to delete pellets upon collision and exit the program if Pac-Man collided with a ghost.

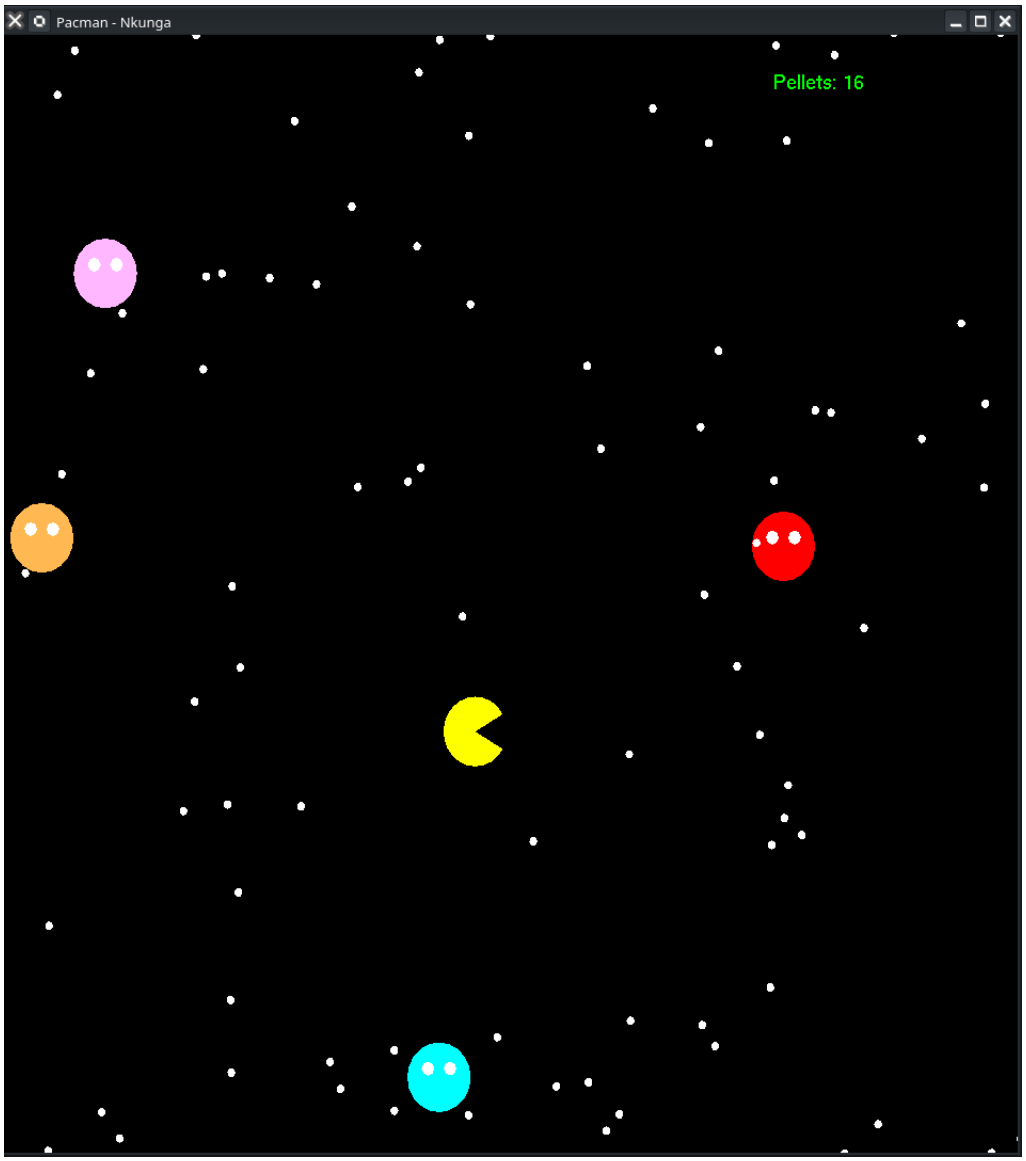
After this came code cleanup and a small amount of embellishment. As implied, I very much mismanaged my time with starting this problem. I had started working on this late into its assignment meaning I couldn't take the time to properly engage with and debug the program on my own, leading to much higher AI usage on a project I was excited to work on. This meant that by this time in the project, main was cluttered with a mess of different functions and the entire codebase was getting harder and harder to read. That being said, wanting to clean the code doesn't automatically give me more time to work so clean up mostly consisted again of telling Clause (my LLM of choice for coding) to separate everything it could from main into separate files.

Embellishment came in the form of the win screen. After discovering the fun of a forced program end with the exit command, I generated some simple ASCII text for game over that changed in color with a win or loss and printed some player states depending on the state of the game.



Testing:

Testing mostly came incrementally. The biggest hurdles came from small mistakes again most becoming so daunting due to the self imposed lack of time. Key standout moments include struggling to realize why my initial attempts at inheritance were failing, accidentally making the pellets on different z-layers meaning I couldn't tell why they weren't colliding with Pac-Man, accidentally discovering how simple screen wrapping would be when trying to implement bouncing for pellets (I accidentally reflected their position instead of speed), and fighting with Claude about how, even with my limited time, I'm still a picky programmer.



Conclusions:

Overall, I'd consider the program a success, but not a great one. If I had more time to fully dive into it, I'd prefer to use less AI, develop different personalities for the ghost akin to those in the original game, create actual ghost sprites, and implement a simple power up to make the game a bit more interesting.

References:

1. How to draw a circle in OpenGL: https://www.reddit.com/r/opengl/comments/6f4q7t/how_do_you_draw_a_circle_in_modern_opengl/
2. Stanford Professor Reacts to Student Coding Projects: https://youtu.be/3mRvCF4qyTA?si=twjMcjSw9mPv2zoP&t=72